WinLocal Giveaways

AWS Migration - Proof of Concept

Architecture design, implementation, additional services and sustainability

Sanitised version for public sharing. Personal identifiers, IP addresses, AWS account details, endpoints and resource identifiers have been removed or masked. Screenshots containing such details have been replaced with placeholders.

Contents

Introduction

Task 1 - Detailed AWS Architecture Diagram (Design Phase)

Task 2 - Proof of Concept (PoC) Implementation and explanation

Task 3 - Justification of Additional AWS Services (Beyond PoC)

Task 4 - Sustainability (AWS Well-Architected Framework)

References

Introduction

As part of the migration from the existing on-premises infrastructure to AWS, WinLocal Giveaways needed a solution that not only improves availability, scalability, and security, but also supports efficient use of cloud resources. The sustainability pillar of the AWS Well-Architected Framework focuses on reducing environmental impact by using resources more efficiently and avoiding unnecessary infrastructure running at all times. In this Proof of Concept, several AWS services such as Auto Scaling, Amazon RDS, and Amazon S3 were selected to help ensure that compute resources are used only when needed. Compared to the previous hosting environment described in the project brief, this cloud-based architecture provides a more flexible and energy-efficient solution. This section evaluates how sustainability principles were applied in the current design and suggests possible improvements that could further support long-term sustainable operation as the WinLocal Giveaways platform continues to expand.

Task 1 - Detailed AWS Architecture Diagram (Design Phase)

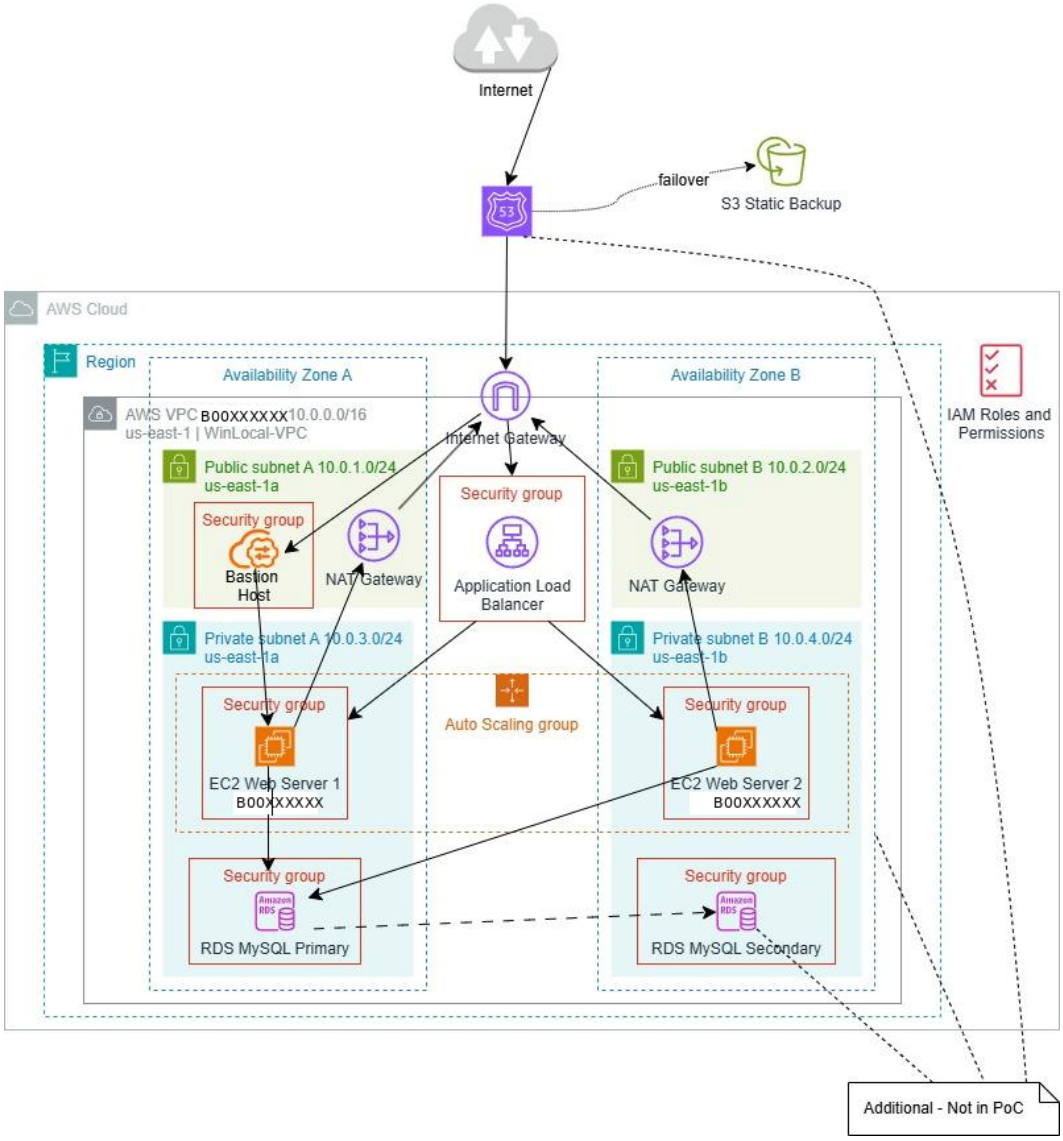


Figure 1 - Target architecture. Components marked 'Additional - Not in PoC' are covered in Task 3.

Task 2 - Proof of Concept (PoC) Implementation and explanation

Networking Design

Screenshot omitted from public version
VPC resource map (resource names contain a student identifier)

The logical network design for WinLocal Giveaways on an AWS environment is made using a Virtual Private Cloud (VPC) with segmented subnets: public and private. This design improves security, scalability, and availability, following AWS best practices architecture principles.

Virtual Private Cloud (VPC)

A project VPC (10.0.0.0/16) was created to provide an isolated network environment for hosting the application infrastructure. The /16 CIDR allows flexible subnet expansion as the platform grows and supports future scaling requirements such as additional application services.

Subnet Segmentation

Two subnets were created in the VPC to separate internet components.

Public Subnet - 10.0.1.0/24 (us-east-1a). Hosts components that must communicate with the internet:

- Bastion Host
- NAT Gateway
- Application Load Balancer

The public subnet routes traffic through the Internet Gateway, enabling controlled inbound and outbound traffic.

Private Subnet - 10.0.3.0/24 (us-east-1a). Hosts backend resources:

- EC2 Web Servers
- RDS MySQL DB

Private subnets do not allow direct internet access, improving protection against external attacks.

Availability Zones

Resources are deployed within a single AZ (us-east-1a), designed to support future multi-AZ, improving fault tolerance, high availability and resilience against AZ failure. If any AZ becomes unavailable, workloads can continue operating in the second zone.

Internet Gateway

It was attached to the VPC to allow communication between public subnet resources and the internet, enabling inbound traffic to the ALB and secure SSH access to the Bastion Host, while private subnet resources remain protected.

NAT Gateway

A NAT Gateway was deployed in the public subnet to allow private subnet instances to access the internet securely. It enables operating system updates, patch downloads, and security updates, while preventing

direct inbound internet access to private EC2 instances.

Bastion Host

Screenshot omitted from public version
Bastion Host instance details and security group rules (contain IP addresses and account ID)

An EC2 instance was configured as a Bastion Host deployed in a public subnet to provide secure administrative access to private EC2 Web Server 1 within the VPC. The bastion host acts as a controlled entry point, ensuring backend servers remain inaccessible directly from the internet.

Screenshot omitted from public version
SSH session from administrator machine, via the Bastion Host, to the private web server

A security group was configured to allow SSH access only from a trusted administrator IP address (in this case the administrator's own IP) on port 22 via the Bastion Host. Applying this rule reduces the risk of unauthorized access attempts to the system.

This design improves network security by reducing the attack surface, preventing direct access of internal servers by any external users except the administrator, who connects first to the bastion host and then securely accesses private instances. It also prevents the previous ransomware attack identified in the brief by enforcing proper network segmentation and controlled access between infrastructure layers.

Amazon Machine Image (AMI)

Screenshot omitted from public version
AMI details (contain account ID and AMI identifiers)

An AMI was created to capture a reusable template of the configured web server instance. The AMI includes the OS, installed software, security settings, and application configurations required to run identical instances fast and consistently.

The AMI enables integration with the Auto Scaling Group so that additional web servers can be launched automatically during periods of high demand.

Using an AMI also increases system reliability and outage recovery capabilities, as replacement instances can be created quickly if a failure occurs. Overall, the AMI supports scalability, resilience, and standardized infrastructure management within the AWS cloud environment.

Single-AZ Amazon RDS Deployment

Screenshot omitted from public version
RDS instance connectivity details and database connection test (contain endpoint, username and IPs)

An Amazon RDS MySQL instance was deployed in the private subnet to securely store users' data. The DB was configured as a Single-AZ deployment for the proof of concept.

Public access to the database was disabled, and connectivity was restricted using a dedicated security group that allows inbound MySQL traffic only from the EC2 web server security group on port 3306, ensuring controlled access in the segmented network.

To check the connectivity between the web application and the RDS instance, a MySQL command-line connection was used and a database test script was executed from the web server, returning a successful connection.

However, a Single-AZ deployment does not provide automatic failover if the Availability Zone becomes unavailable. To improve availability and resilience, the architecture could be improved by implementing a Multi-AZ RDS deployment, which maintains a secondary RDS in a second Availability Zone and enables automatic failover during outages.

Auto Scaling Configuration

Screenshot omitted from public version
Auto Scaling group capacity overview and target tracking policy (contain account ID in ARN)

An Amazon EC2 Auto Scaling Group was configured to automatically manage the number of web server instances based on application demand (desired capacity 1, scaling limits 1-2). The ASG was created using a launch template from the previously configured AMI, ensuring that newly launched instances maintain consistent operating system, software and security configurations. The scaling policy was configured as a target tracking policy to maintain an average CPU utilization across all instances at 60%, automatically increasing and decreasing the number of instances during variable periods, supporting both performance and cost optimization requirements defined by the client.

Screenshot omitted from public version
Auto Scaling activity history (contains instance identifiers)

The activity history shows instances being launched when capacity was increased and replaced when health checks failed. This evidence shows that the scaling policy operates correctly and supports dynamic workload management in the cloud environment.

Load Balancer configuration

Screenshot omitted from public version
Application Load Balancer resource map and test responses from two web servers (contain DNS names and identifiers)

An Application Load Balancer was deployed within the public subnet to distribute HTTP traffic to two EC2 instances in the private subnet. The load balancer was configured with an HTTP listener on port 80 and associated with a target group containing instances launched by the Auto Scaling Group.

Traffic distribution was verified by accessing the load balancer DNS endpoint and observing responses from two different EC2 instances (WinLocal Web Server 1 and WinLocal Web Server 2). This confirms that incoming traffic is routed dynamically between instances, demonstrating correct load balancing functionality.

The Application Load Balancer is important for system availability and reliability by preventing a single EC2 instance from being overused and crashing. It ensures that traffic is distributed automatically across multiple instances during increased utilization and allows integration with Auto Scaling so new instances are automatically deployed. This supports scalability, fault tolerance, and consistent service delivery for the WinLocal Giveaways platform.

Security Implementation

Screenshot omitted from public version
Security groups list (contains security group and VPC identifiers)

Security groups were implemented to enforce security and controlled communication between different infrastructure layers in the AWS environment. Having two separated subnets (public and private), different security groups were created for dedicated instances.

- **RDS Security Group:** permits MySQL traffic only from the web server security group on port 3306.
- **Application Load Balancer Security Group:** allows inbound HTTP traffic from users and forwards requests to the EC2 web servers.
- **Bastion Security Group:** allows SSH access only from a trusted administrator IP address on port 22, ensuring secure administrative access without exposing private instances to the internet.
- **Web Server Security Group:** restricts inbound traffic to HTTP from the load balancer and SSH access only from the Bastion Host.

Static Backup Website

Screenshot omitted from public version
S3 bucket object list and static website hosting settings (contain bucket name and endpoint)

A static backup website was deployed using Amazon S3 static website hosting to provide an alternative display message when the primary instance becomes unavailable. The S3 bucket was configured with static website hosting and an index page that displays a service availability message to users during outages.

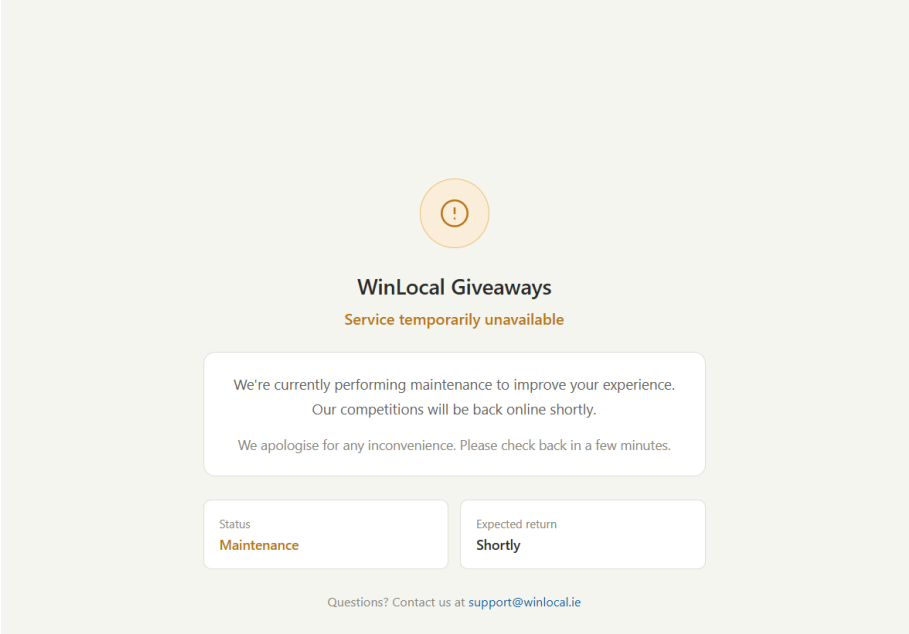


Figure 2 - Static failover page served from Amazon S3 (browser address bar cropped).

The static website can be accessed directly using the bucket website endpoint URL, which confirms that the failover page is publicly available.

This backup bucket would be used when the primary instances become unavailable due to system failure, maintenance events, or security issues. In full production, Amazon Route 53 DNS failover routing could automatically redirect users to this static backup site, ensuring continued service availability and improving overall platform resilience and user experience during outages.

Task 3 - Justification of Additional AWS Services (Beyond PoC)

Service 1 - Amazon Route 53 with Health Check and DNS Failover

Amazon Route 53 is proposed to solve the first issue, where periodic load balancer failures make the WinLocal website completely inaccessible. To resolve this, an automatic DNS-based failover mechanism is required that redirects users to a static backup webpage hosted on Amazon S3 when the primary application becomes unavailable.

Route 53 is AWS's managed Domain Name System service that translates domain names into IP addresses and supports health check-based failover routing. In this architecture, Route 53 continuously monitors the health of the Application Load Balancer by sending periodic health check requests. If the load balancer becomes unreachable after multiple failed checks, Route 53 automatically redirects traffic to the S3 static backup website, ensuring users still receive service availability information instead of an error page.

To implement this solution, a hosted zone would be created for the WinLocal domain, with a primary DNS record pointing to the load balancer and a secondary failover record pointing to the S3 backup site. This configuration improves service resilience, maintains user trust, and ensures continuity during infrastructure failures.

Service 2 - Multi-AZ RDS Deployment (Availability Zone B)

Screenshot omitted from public version
VPC resource map showing public and private subnets in us-east-1a and us-east-1b

A second Availability Zone (us-east-1b) will solve the second issue mentioned in the brief (Database Reliability and on-premises Storage Limitations), where the database server frequently runs out of storage, resulting in downtime and loss of access to competition entry data.

Multi-AZ RDS automatically provides and maintains a secondary database in a second Availability Zone. Every change made in the database is synchronously replicated to the standby before being confirmed. If the primary database fails, AWS will automatically fail over to the second AZ within approximately 60 seconds.

It can be configured for WinLocal by modifying the existing Single-AZ RDS instance and enabling the Multi-AZ option in the RDS console. AWS will automatically create a secondary replica in the private subnet in us-east-1b. The DB Subnet Group already covers both private subnets, so no networking changes are required. The same Security Group will be applied to both instances.

Task 4 - Sustainability (AWS Well-Architected Framework)

The sustainability pillar of the AWS Well-Architected Framework focuses on improving resource efficiency by ensuring that cloud infrastructure is only used when required. As WinLocal Giveaways migrates from its previous on-premises hosting environment to AWS, the new architecture supports more efficient use of computing resources while maintaining scalability and availability.

In this Proof of Concept, Auto Scaling helps reduce unnecessary resource usage by running only one EC2 instance during normal operation and automatically launching an additional instance only when demand

increases. This ensures that infrastructure is not lacking in scalability. The use of Amazon S3 static website hosting for the backup failover page also improves efficiency, as it removes the need to keep additional standby servers running continuously. In addition, deploying services such as EC2 and RDS within private subnets supports controlled and efficient network communication between instances.

For future improvements, WinLocal could optimize resource usage by implementing Amazon Route 53 DNS failover routing together with Multi-AZ deployments for critical services such as Amazon RDS and EC2 Auto Scaling. Route 53 health checks will automatically redirect users to the static backup website hosted on Amazon S3 if the primary infrastructure becomes unavailable. Deploying resources across multiple Availability Zones would also improve reliability while supporting efficient workload distribution during traffic outages.

In addition, Amazon CloudWatch could be used to monitor infrastructure utilization and identify unused or underutilized resources so they can be resized or removed when not required. These improvements would help WinLocal maintain efficient resource usage, reduce unnecessary infrastructure overhead, and support sustainable and resilient long-term platform growth as user demand increases.

References

AWS (n.d.) *What is Amazon VPC?* Available at: https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html

AWS (n.d.) *Get started with Amazon EC2 Auto Scaling.* Available at: https://docs.aws.amazon.com/autoscaling/ec2/userguide/get-started-with-ec2-auto-scaling.html

AWS (2023) *Drive application performance with Application Load Balancer Target Optimizer.* Available at: https://aws.amazon.com/blogs/networking-and-content-delivery/drive-application-performance-with-application-load-balancer-target-optimizer/

AWS (n.d.) *Load balancer CloudWatch logs.* Available at: https://docs.aws.amazon.com/elasticloadbalancing/latest/network/load-balancer-cloudwatch-logs.html

AWS (n.d.) *AWS Academy Labs.* Available at: https://aws.amazon.com/training/awsacademy/